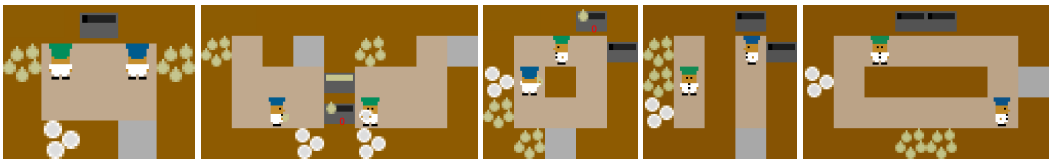


Figure 3: Experiment layouts. From left to right: *Cramped Room* presents low-level coordination challenges: in this shared, confined space it is very easy for the agents to collide. *Asymmetric Advantages* tests whether players can choose high-level strategies that play to their strengths, as illustrated in Figure 2. In *Coordination Ring*, players must coordinate to travel between the bottom left and top right corners of the layout. *Forced Coordination* instead removes collision coordination problems, and forces players to develop a high-level joint strategy, since neither player can serve a dish by themselves. *Counter Circuit* involves a non-obvious coordination strategy, where onions are passed over the counter to the pot, rather than being carried around.

Behavior cloning. One of the simplest approaches to imitation learning is given by behavior cloning, which learns a policy from expert demonstrations by directly learning a mapping from observations to actions with standard supervised learning methods [4]. Since we have a discrete action space, this is a traditional classification task. Our model takes an encoding of the state as input, and outputs a probability distribution over actions, and is trained using the standard cross-entropy loss function.

Population Based Training. Population Based Training (PBT) [19] is an online evolutionary algorithm which periodically adapts training hyperparameters and performs model selection. In multiagent RL, PBT maintains a population of agents, whose policies are parameterized by neural networks, and trained with a DRL algorithm. In our case, we use Proximal Policy Optimization (PPO) [32]. During each PBT iteration, pairs of agents are drawn from the population, trained for a number of timesteps, and have their performance recorded. At the end of each PBT iteration, the worst performing agents are replaced with copies of the best agents with mutated hyperparameters.

4 Environment and Agents

4.1 Overcooked

To test our hypotheses, we would like an environment in which coordination is challenging, and where deep RL algorithms work well. Existing environments have not been designed to be challenging for coordination, and so we build a new one based on the popular video game *Overcooked* [13], in which players control chefs in a kitchen to cook and serve dishes. Each dish takes several high-level actions to deliver, making strategy coordination difficult, in addition to the challenge of motion coordination.

We implement a simplified version of the environment, in which the only objects are onions, dishes, and soups (Figure 2). Players place 3 onions in a pot, leave them to cook for 20 timesteps, put the resulting soup in a dish, and serve it, giving all players a reward of 20. The six possible actions are: up, down, left, right, noop, and "interact", which does something based on the tile the player is facing, e.g. placing an onion on a counter. Each layout has one or more onion dispensers and dish dispensers, which provide an unlimited supply of onions and dishes respectively. Most of our layouts (Figure 3) were designed to lead to either low-level motion coordination challenges or high-level strategy challenges.

Agents should learn how to navigate the map, interact with objects, drop the objects off in the right locations, and finally serve completed dishes to the serving area. All the while, agents should be aware of what their partner is doing and coordinate with them effectively.

4.2 Human models

We created a web interface for the game, from which we were able to collect trajectories of humans playing with other humans for the layouts in Figure 3. In preliminary experiments we found that human models learned through behavior cloning performed better than ones learned with Generative Adversarial Imitation Learning (GAIL) [16], so decided to use the former throughout our experiments. To incentivize generalization in spite of the scarcity of human data, we perform behavior cloning over a manually designed featurization of the underlying game state.

For each layout we gathered ~16 human-human trajectories (for a total of 18k environment timesteps). We partition the joint trajectories into two subsets, and split each trajectory into two single-agent trajectories. For each layout and each subset, we learn a human model through behavior cloning. We treat one model, BC, as a human model we have access to, while the second model H_{Proxy} is treated as the ground truth human proxy to evaluate against at test time. On the first three layouts, when paired with themselves, most of these models perform similarly to an average human. Performance is significantly lower for the last two layouts (Forced Coordination and Counter Circuit).

The learned models sometimes got "stuck": they would perform the same action over and over again (such as walking into each other), without changing the state. We added a simple rule-based mechanism to get the agents unstuck by taking a random action. For more details see Appendix A.

4.3 Agents designed for self-play

We consider two DRL algorithms that train agents designed for self-play, and one planning algorithm.

DRL algorithms. For DRL we consider PPO trained in self-play (SP) and PBT. Since PBT involves training agents to perform well with a *population* of other agents, we might expect them to be more robust to potential partners compared to agents trained via self-play, and so PBT might coordinate better with humans. For PBT, all of the agents in the population used the same network architecture.

Planning algorithm. Working in the simplified Overcooked environment enables us to also compute near-optimal plans for the joint planning problem of delivering dishes. This establishes a baseline for performance and coordination, and is used to perform *coupled planning with re-planning*. With coupled planning, we compute the optimal joint plan for both agents. However, rather than executing the full plan, we only execute the first action of the plan, and then *replan* the entire optimal joint plan after we see our partner's action (since it may not be the same as the action we computed for them).

To achieve this, we pre-compute optimal joint motion plans for every possible starting and desired goal locations for each agent. We then create high-level actions such as "get an onion", "serve the dish", etc. and use the motion plans to compute the cost of each action. We use A^* search to find the optimal joint plan in this high-level action space. This planner does make some simplifying assumptions, detailed in Appendix E, making it near-optimal instead of optimal.

4.4 Agents designed for humans

We seek the simplest possible solution to having an agent that actually takes advantage of the human model. We embed our learned human model BC in the environment, treating it's choice of action as part of the dynamics. We then directly train a policy on this single-agent environment with PPO. We found empirically that the policies achieved best performance by initially training them in self-play, and then linearly annealing to training with the human model (see Appendix C).

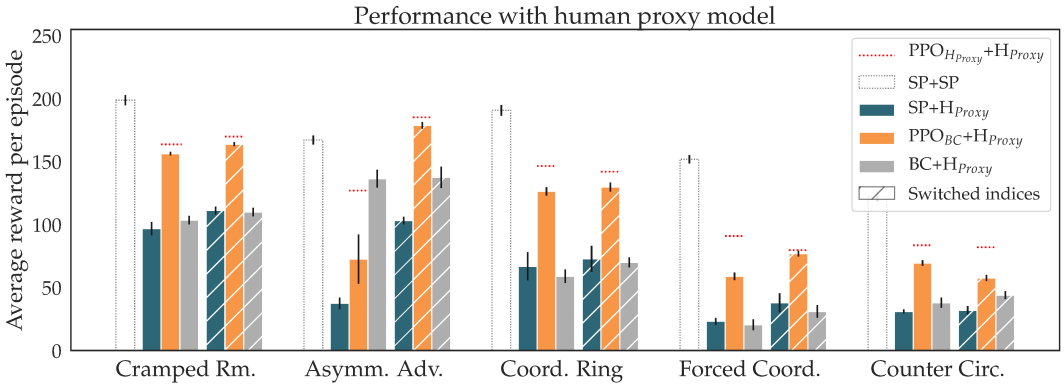
For planning, we implemented a model-based planner that uses hierarchical A^* search to act near-optimally, assuming access to the policy of the other player (see Appendix F). In order to preserve near-optimality we make our training and test human models deterministic, as dealing with stochasticity would be too computationally expensive.

5 Experiments in Simulation

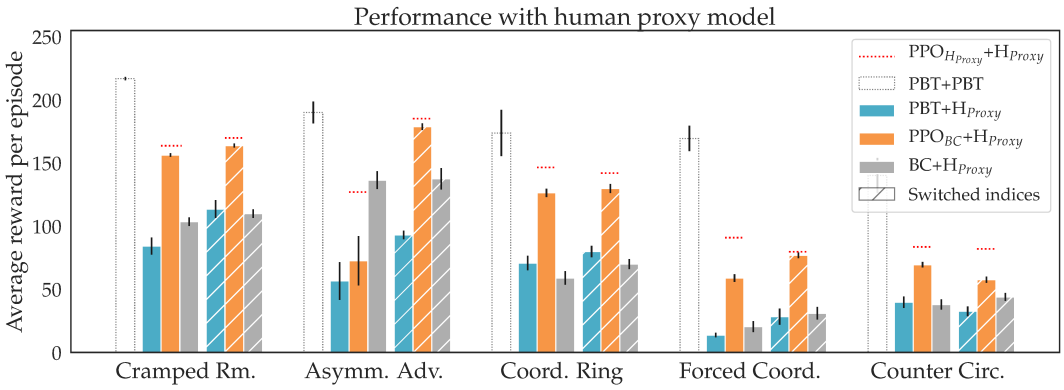
We pair each agent with the proxy human model and evaluate the team performance.

Independent variables. We vary the type of agent used. We have agents trained with themselves: self-play (SP), population-based training (PBT), coupled planning (CP); agents trained with the human model BC: reinforcement learning based (PPO$_{BC}$), and planning-based P$_{BC}$; and the imitation agent BC itself. Each agent is paired with H$_{Proxy}$ in each of the layouts in Figure 3.

Dependent measures. As good coordination between teammates is essential to achieve high returns in this environment, we use cumulative rewards over a horizon of 400 timesteps for our agents as a proxy for coordination ability. For all DRL experiments, we report average rewards across 100 rollouts and standard errors across 5 different seeds. To aid in interpreting these measures, we also compute a "gold standard" performance by training the PPO and planning agents not with BC, but with H$_{Proxy}$ itself, essentially giving them access to the ground truth human they will be paired with.



(a) Comparison with agents trained in self-play.



(b) Comparison with agents trained via PBT.

Figure 4: Rewards over trajectories of 400 timesteps for the different agents (agents trained with themselves – SP or PBT – in teal, agents trained with the human model – PPO$_{BC}$ – in orange, and imitation agents – BC – in gray), with standard error over 5 different seeds, paired with the proxy human H$_{Proxy}$. The white bars correspond to what the agents trained with themselves *expect* to achieve, i.e. their performance when paired with itself (SP+SP and PBT+PBT). First, these agents perform much worse with the proxy human than with themselves. Second, the PPO agent that trains with human data performs much better, as hypothesized. Third, imitation tends to perform somewhere in between the two other agents. The red dotted lines show the "gold standard" performance achieved by a PPO agent with direct access to the proxy model itself – the difference in performance between this agent and PPO$_{BC}$ stems from the innacuracy of the BC human model with respect to the actual H_{Proxy}. The hashed bars show results with the starting position of the agents switched. This most makes a difference for asymmetric layouts such as Asymmetric Advantages or Forced Coordination.

Analysis. We present quantitative results for DRL in Figure 4. Even though SP and PBT achieve excellent performance in self-play, when paired with a human model they struggle to even meet the performance of the imitation agent. There is a large gap between the imitation agent and gold standard performance. Note that the gold standard reward is lower than self-play methods paired with themselves, due to human suboptimality affecting the highest possible reward the agent can get. We then see that PPO$_{BC}$ outperforms the agents trained with themselves, getting closer to the gold standard. This supports our hypothesis that 1) self-play-like agents perform drastically worse when paired with humans, and 2) it is possible to improve performance significantly by taking the human into account. We show this holds (for these layouts) even when using an unsophisticated, behavior-cloning based model of the human.

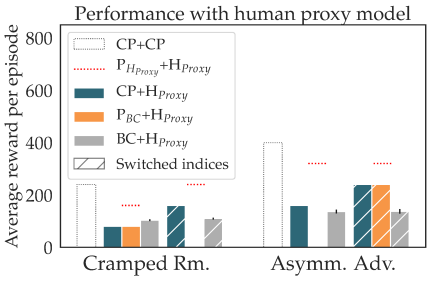


Figure 5: Comparison across planning methods. We see a similar trend: coupled planning (CP) performs well with itself (CP+CP) and worse with the proxy human (CP+H$_{Proxy}$). Having the correct model of the human (the dotted line) helps, but a bad model (P$_{BC}$+H$_{Proxy}$) can be much worse because agents get stuck (see Appendix G).